Estruturas de Dados Básicas II

Prof. Dr. Matheus da Silva Menezes



UFRN
UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Começando a Unidade 02.

Estruturas de Dados Básicas II

HEAPSORT
Conceitos Iniciais


HEAPS
O QUE SÃO, ONDE VIVEM,
DE QUE SE ALIMENTAM
SAIBA NA AULA DE HOJE!

Definição

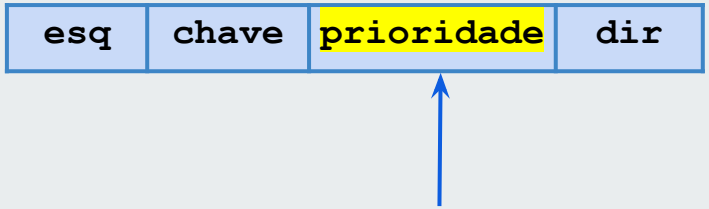
HEAP = Amontoar (tradução)







Uma representação para nó de uma árvore binária / heap



Exemplo C

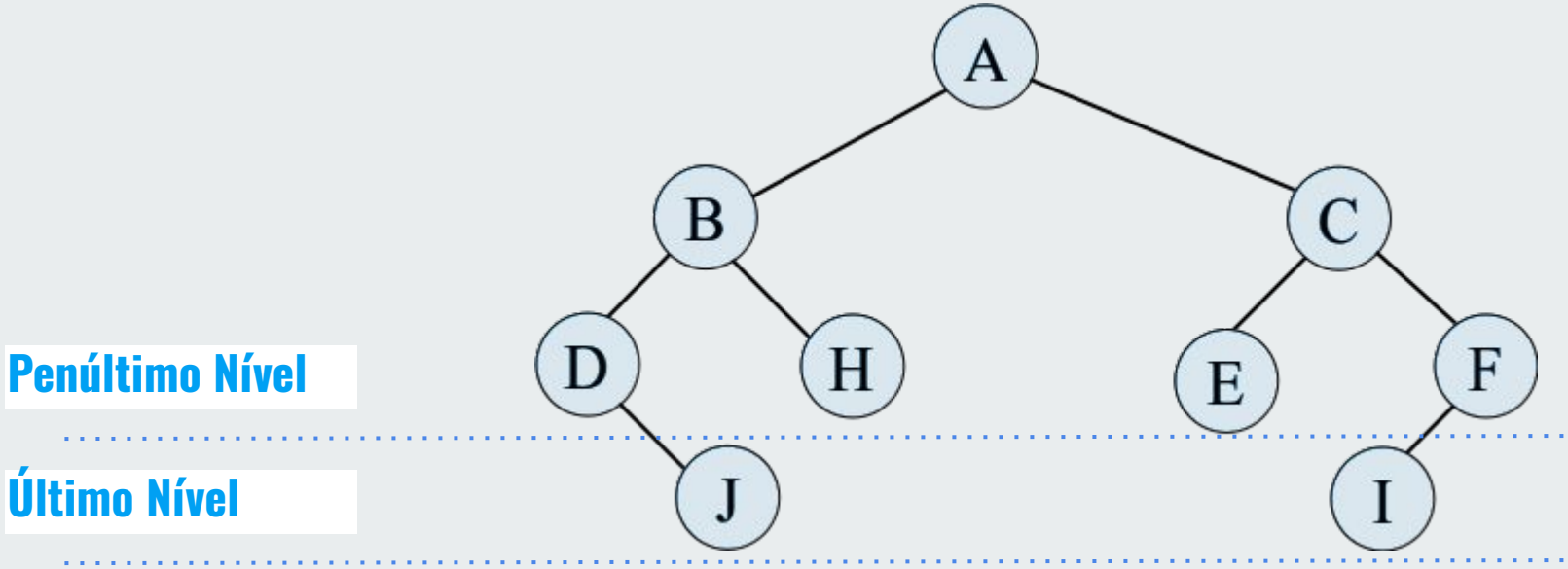
```
struct nodo{
  int chave;
  int prioridade;
  struct nodo *esq;
  struct nodo *dir; };
typedef struct nodo no;
}
```

```
typedef no *pont_no;
```

Observação: O campo chave será usado como chave e conteúdo/prioridade

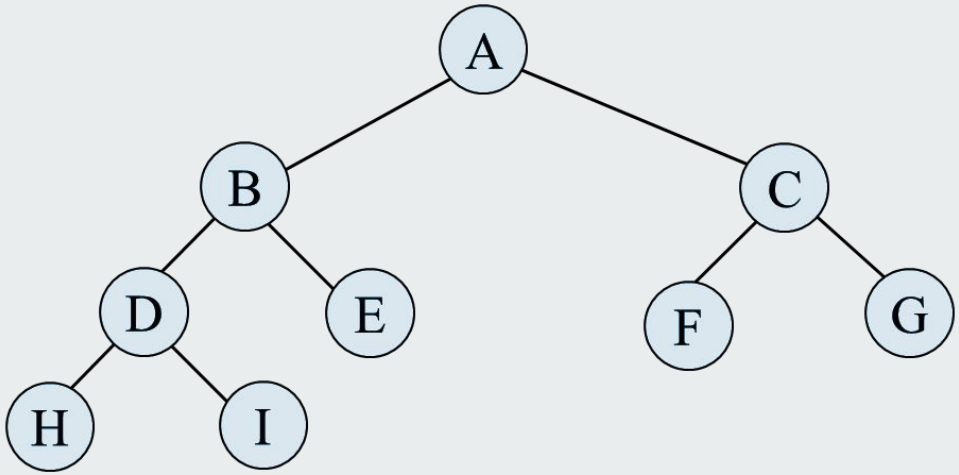
Árvore Binária Completa

Se `v` *é um nó com uma subárvore vazia, então* `v` *está no último ou no penúltimo nível de* `T`.



Representação Vetorial de uma Árvore Binária Completa

Caso especial onde o último nível é preenchido ***da esquerda para a direita***



Filho esquerdo do nó na posição i :
posição `2i`

Filho direito do nó na posição i :
posição `2i + 1`

Pai do nó na posição i :
posição `⌊i/2⌋`

***n*= 9** ***max* = 15**

A	B	C	D	E	F	G	H	I						
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Representação Vetorial de uma Árvore Binária Completa

*Caso especial onde o último nível é preenchido **da esquerda para a direita***

a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}	a_{12}

Filho esquerdo do nó na posição i :
posição `2i`

Filho direito do nó na posição i :
posição `2i + 1`

Pai do nó na posição i :
posição $\lfloor i/2 \rfloor$

Consegue visualizar os elementos da subárvore no vetor?

Definição

- **Uma lista de prioridade é uma tabela na qual a cada um de seus dados está associada uma prioridade.**
- **Uma estrutura de dados heap é uma lista linear que pode ser visualizada como uma árvore binária completa, com o último nível preenchido da esquerda para a direita.**



Definição

Uma lista de prioridade é uma lista de elementos com prioridades $s_1, \ldots, s_n$, satisfazendo a seguinte propriedade:

MAX

$s_i \leq s_{\lfloor i/2 \rfloor}$ para $1 \leq i \leq n$

MIN

$s_i \geq s_{\lfloor i/2 \rfloor}$ para $1 \leq i \leq n$



Definição

Propriedade de Heap:

- *Em um Heap, todo nó deve ter prioridade maior (menor) do que a dos seus filhos (caso existam)*



Definição

Propriedade de Heap:

- *Em um Heap, todo nó deve ter prioridade maior (menor) do que a dos seus filhos (caso existam)*

MAX

$s_i \leq s_{\lfloor i/2 \rfloor}$ para $1 \leq i \leq n$

16	14	10	8	7	9	3	2	4	1



Definição

Propriedade de Heap:

- *Em um Heap, todo nó deve ter prioridade maior (menor) do que a dos seus filhos (caso existam)*

MIN

$$s_i \geq s_{\lfloor i/2 \rfloor} \text{ para } 1 \leq i \leq n$$

10	14	20	17	27	21	25	19	40	31



`H` é o vetor que armazena a heap.

A raiz da árvore é `H[1]`. Dado `i` o índice de um nó, os índices do seu pai e filhos são dados por:

- `Pai(i) = i div 2`
- `Filho_esq(i) = 2i`
- `Filho_dir(i) = 2i + 1`



As listas a seguir são heaps?

20	25	32	29	27	35	40	45

20	32	25	29	27	35	40	45

50	45	48	29	15	35	40	27

Operações Básicas

1. **ALTERAÇÃO DE PRIORIDADE**
2. **INSERÇÃO**
3. **REMOÇÃO**
4. **CONSTRUÇÃO**



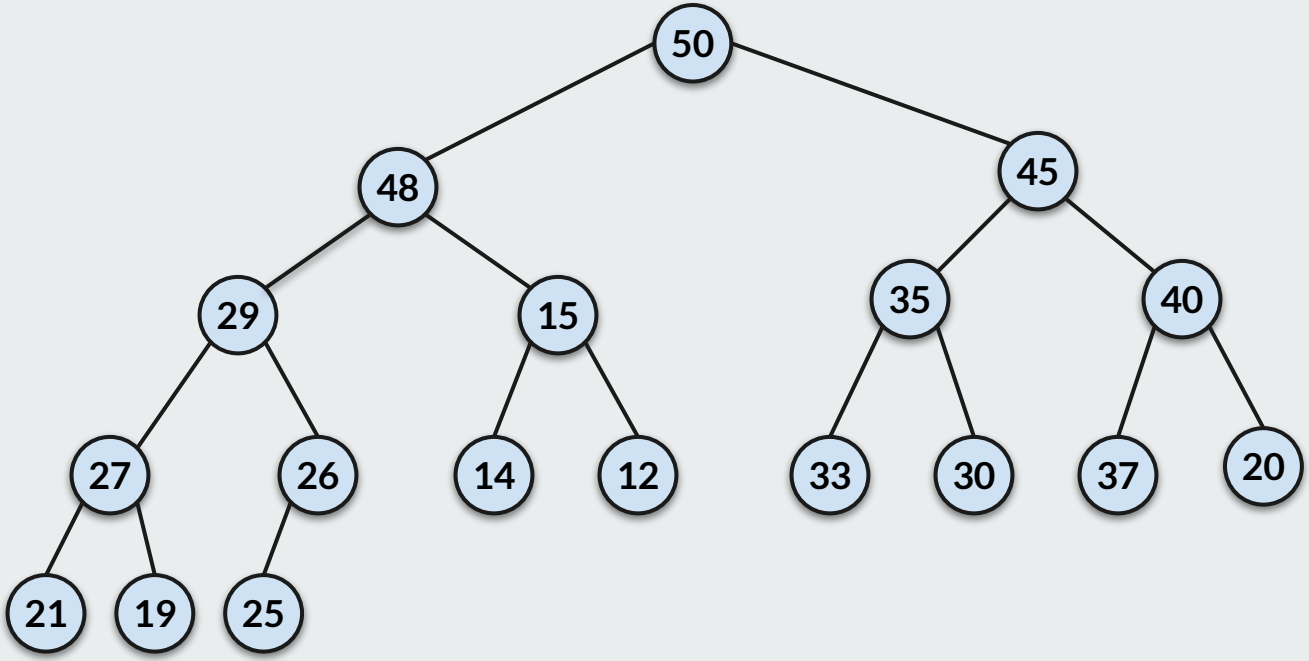
Operações Básicas

1. **ALTERAÇÃO DE PRIORIDADE**
2. **INSERÇÃO**
3. **REMOÇÃO**
4. **CONSTRUÇÃO**



Alteração de Prioridade

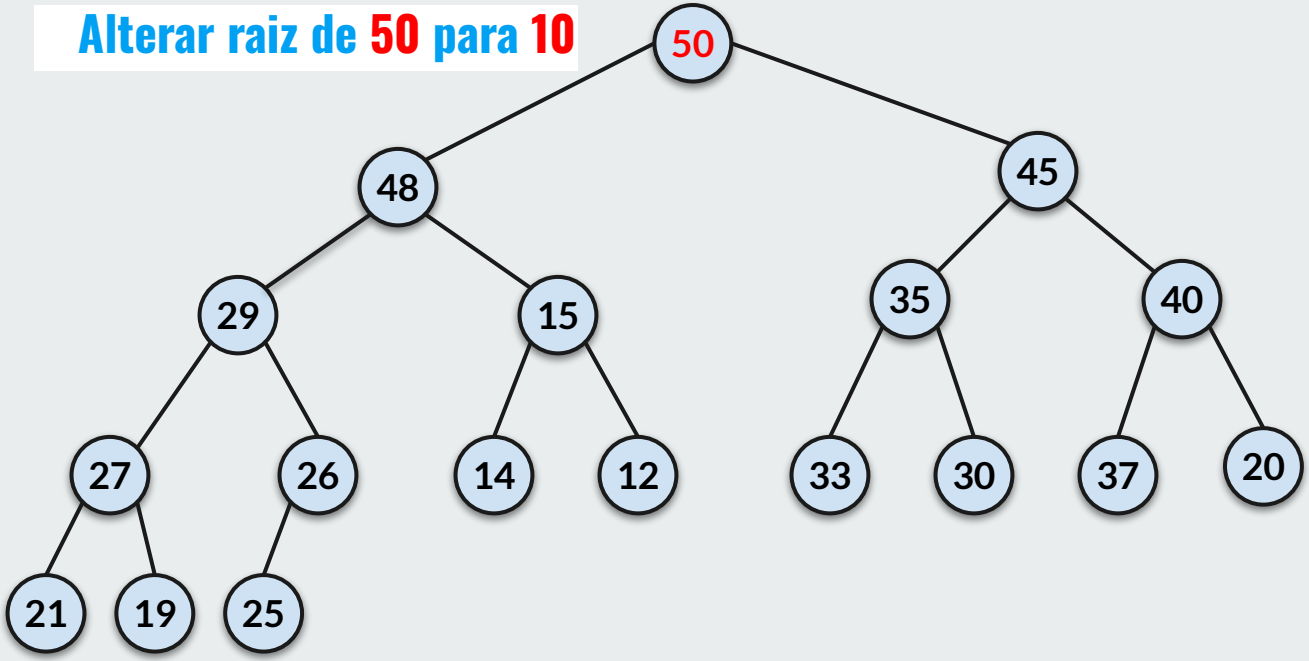
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
50	48	45	29	15	35	40	27	26	14	12	33	30	37	20	21	19	25



Alteração de Prioridade

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
50	48	45	29	15	35	40	27	26	14	12	33	30	37	20	21	19	25

Alterar raiz de 50 para 10



Alteração de Prioridade

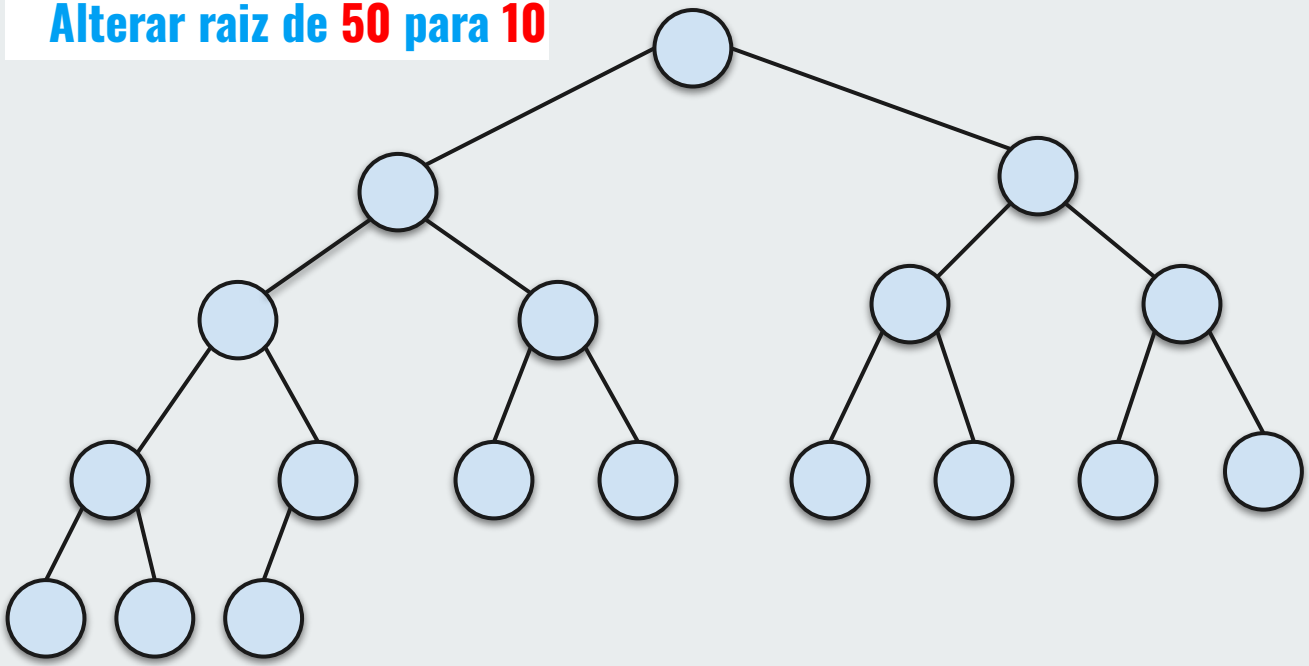
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
50	48	45	29	15	35	40	27	26	14	12	33	30	37	20	21	19	25

Alterar raiz de 50 para 10

Alteração de Prioridade

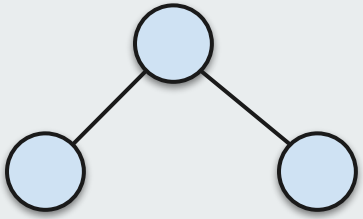
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
50	48	45	29	15	35	40	27	26	14	12	33	30	37	20	21	19	25

Alterar raiz de 50 para 10



Pseudocódigo

```
descer(lista H, int i){
    maior = i;
    se (2*i<= H.tamanho e H[2*i]< H[maior]))
        maior = 2*i; //Se o filho da esquerda é maior, atualiza
    se (2*i+1<= H.tamanho e H[2*i+1]< H[maior]))
        maior = 2*i+1; //Se o filho da direita é maior, atualiza
    se (maior != i){
        troca(H[i],H[maior]); //troca raiz com maior dos filhos
       descer(H, maior);
    }
}
```





Definição

A função `descer(lista H, int i)`:

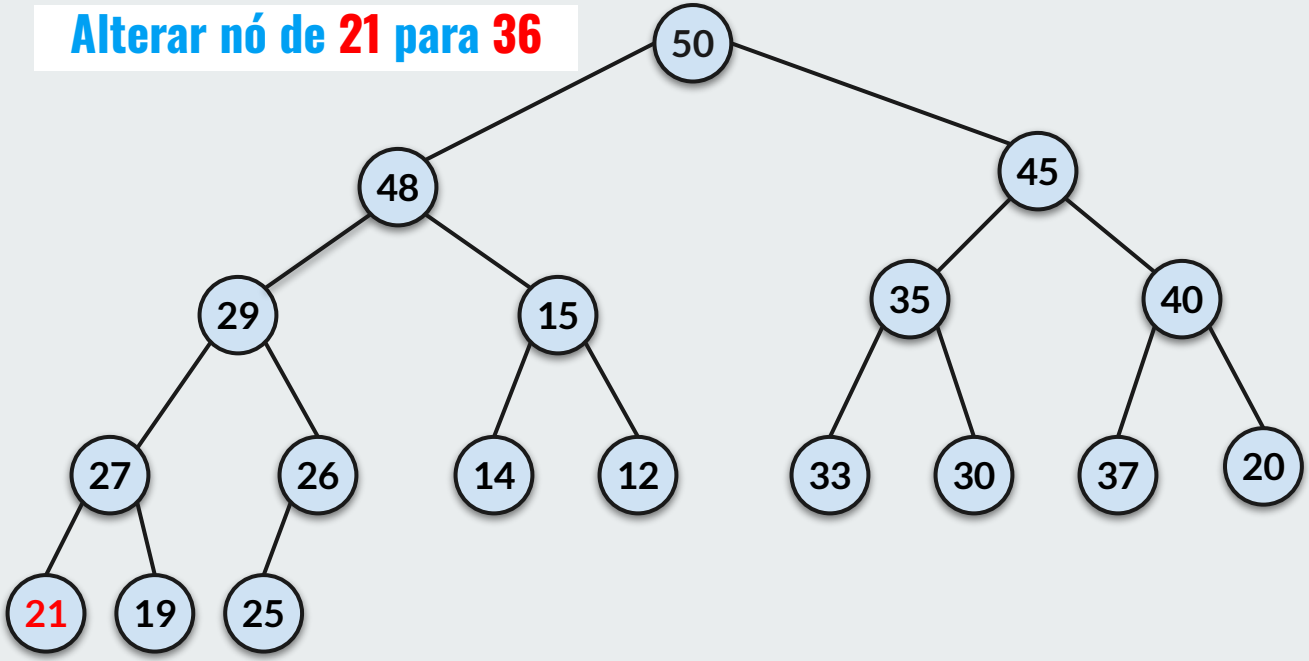
- *Considera que as sub-árvores esquerda e direita são heaps;*
- *Transforma a árvore enraizada em `i` em uma heap;*
- *Qual a complexidade da função descer?*



Alteração de Prioridade

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
50	48	45	29	15	35	40	27	26	14	12	33	30	37	20	21	19	25

Alterar nó de 21 para 36



Alteração de Prioridade

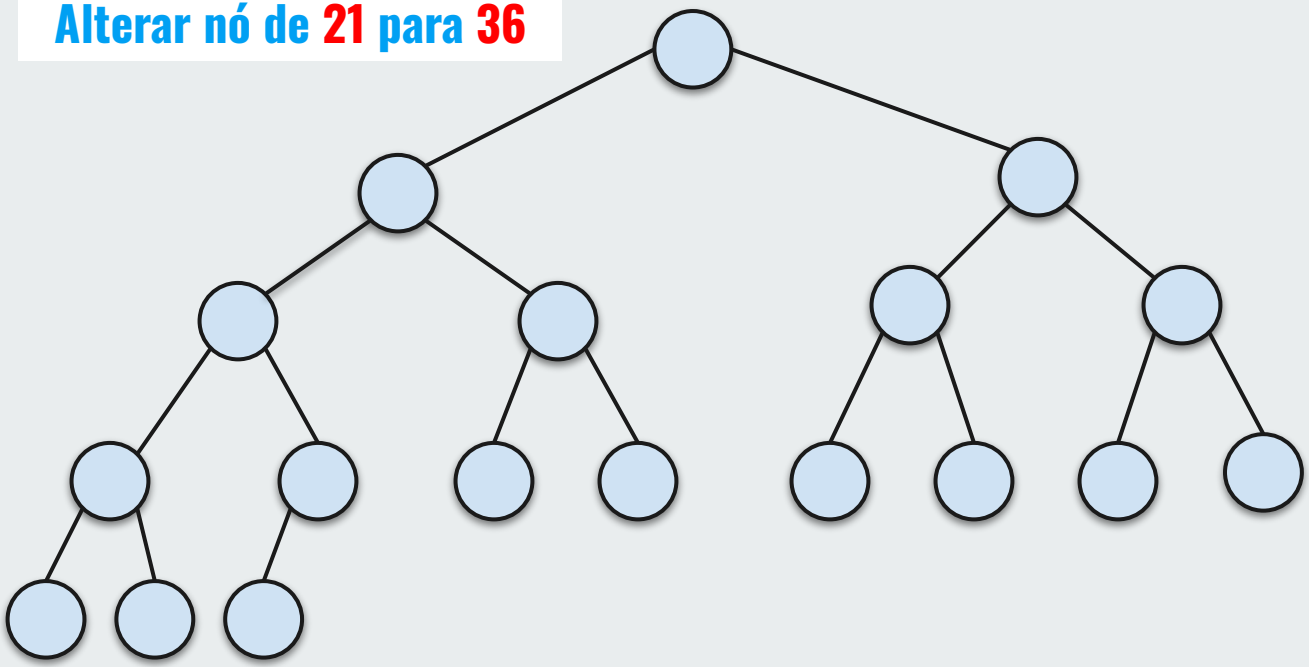
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
50	48	45	29	15	35	40	27	26	14	12	33	30	37	20	21	19	25

Alterar nó de 21 para 36

Alteração de Prioridade

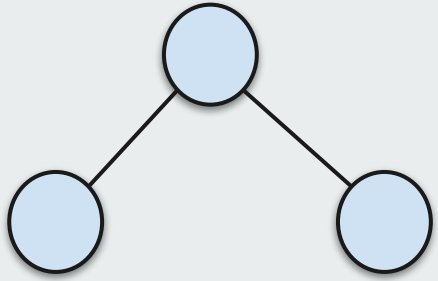
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
50	48	45	29	15	35	40	27	26	14	12	33	30	37	20	21	19	25

Alterar nó de 21 para 36



Pseudocódigo

```
subir(lista H, int i){
    pai = ⌊i/2⌋;
    se (i >1 e H[i]> H[pai])){
      troca(H[i],H[pai]); //troca nó com pai
      subir(H, pai);
    }
}
```





Definição

A função `subir(lista H, int i)`:

- *Considera que `H[1,...i-1]` é heap;*
- *Transforma `H[1,...i1]` em uma heap;*



Operações Básicas

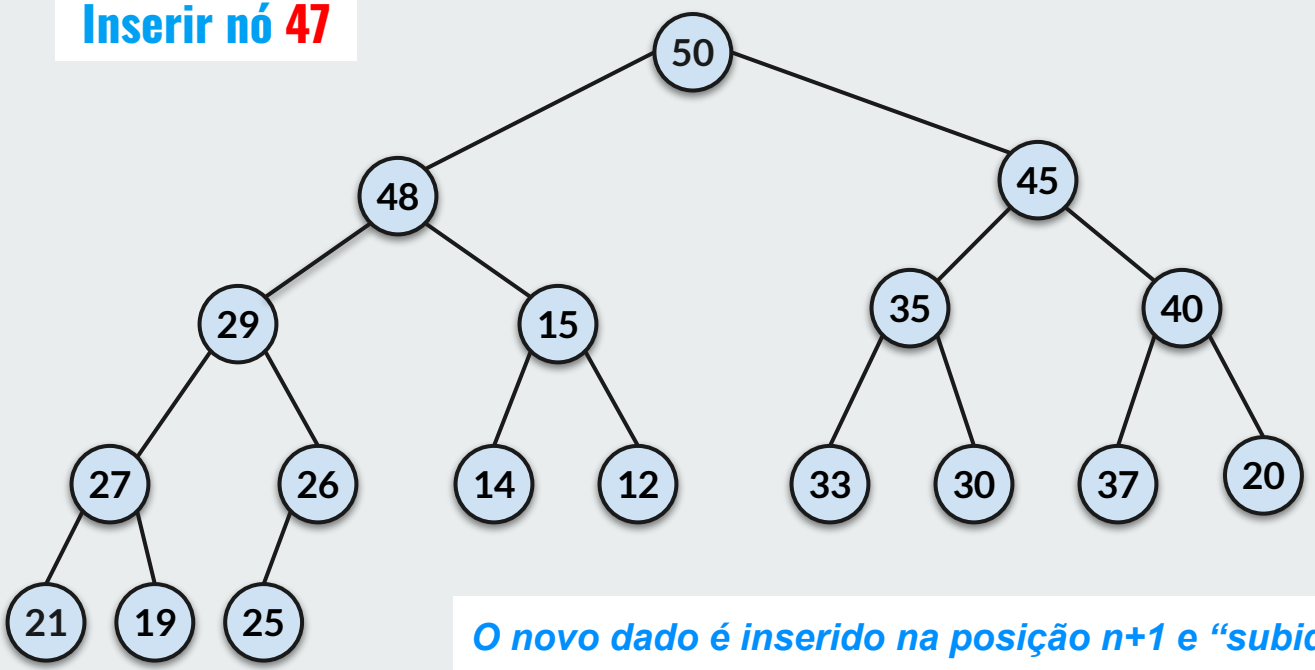
1. ALTERAÇÃO DE PRIORIDADE
2. **INSERÇÃO**
3. REMOÇÃO
4. CONSTRUÇÃO



Inserir novo nó

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
50	48	45	29	15	35	40	27	26	14	12	33	30	37	20	21	19	25

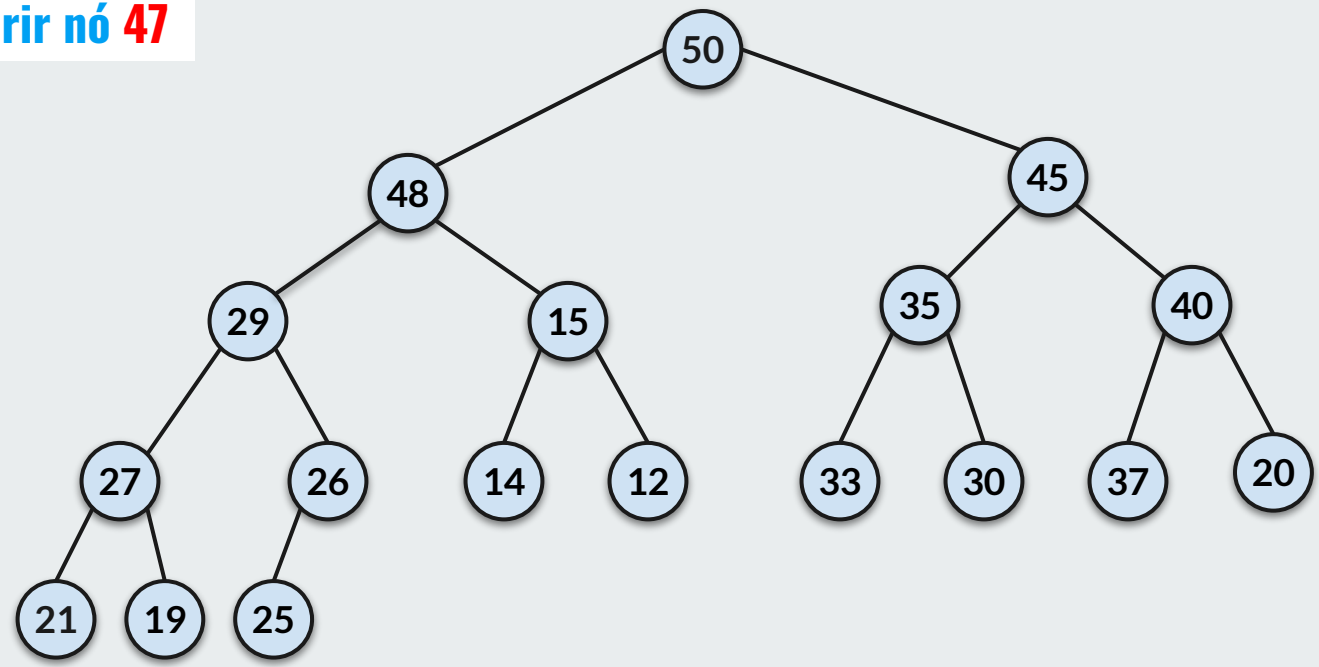
Inserir nó 47



O novo dado é inserido na posição n+1 e "subido" se necessário

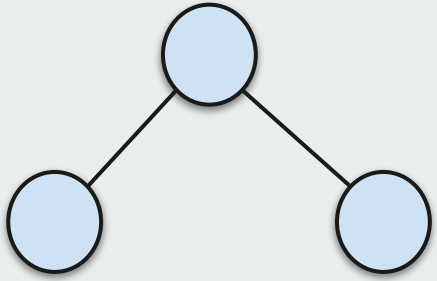
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	
50	48	45	29	15	35	40	27	26	14	12	33	30	37	20	21	19	25	

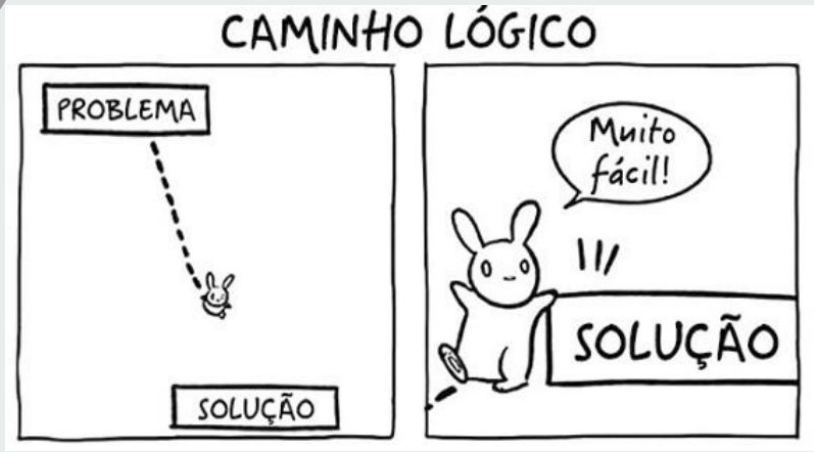
Inserir nó 47



Pseudocódigo

```
inserir(lista H, int x){
    H.tamanho = H.tamanho+1;
    x = H[H.tamanho];
    subir(H, H.tamanho);
}
```





Operações Básicas

1. ALTERAÇÃO DE PRIORIDADE
2. INSERÇÃO
3. **REMOÇÃO**
4. CONSTRUÇÃO



Operações Básicas

1. ALTERAÇÃO DE PRIORIDADE
2. INSERÇÃO
3. REMOÇÃO
4. CONSTRUÇÃO

REMOÇÃO

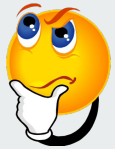
Em um heap binário convencional, a exclusão é geralmente realizada apenas na raiz (ou seja, o maior elemento em um max-heap ou o menor em um min-heap).

*Excluir um elemento arbitrário exigiria reposicionar os elementos para manter a árvore binária completa e realizar uma série de operações de **"heapificação"**, o que seria ineficiente.*

Uma opção é marcar elementos como "removidos" em vez de realmente excluí-los.

Remoção

E como fazemos isso sem muito prejuízo à estrutura da heap?



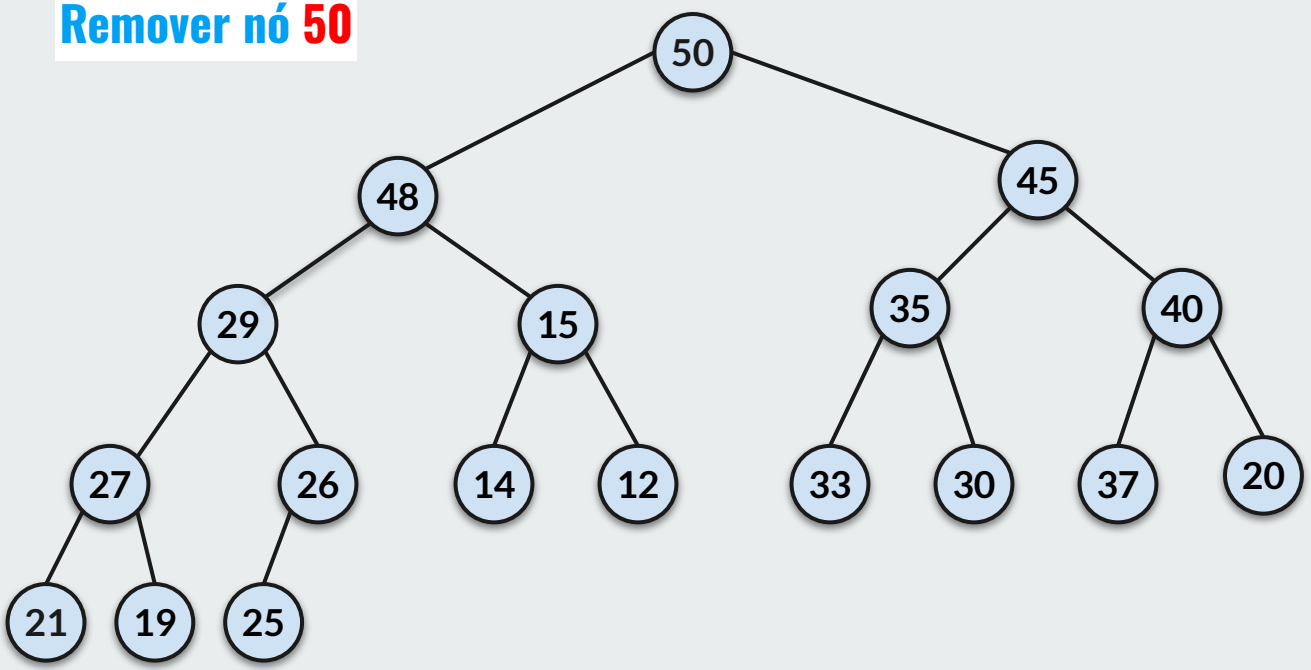
O último elemento substitui o primeiro e será usado o procedimento ***descer*** *para ajustar este elemento na heap novamente.*



Remover raiz

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
50	48	45	29	15	35	40	27	26	14	12	33	30	37	20	21	19	25

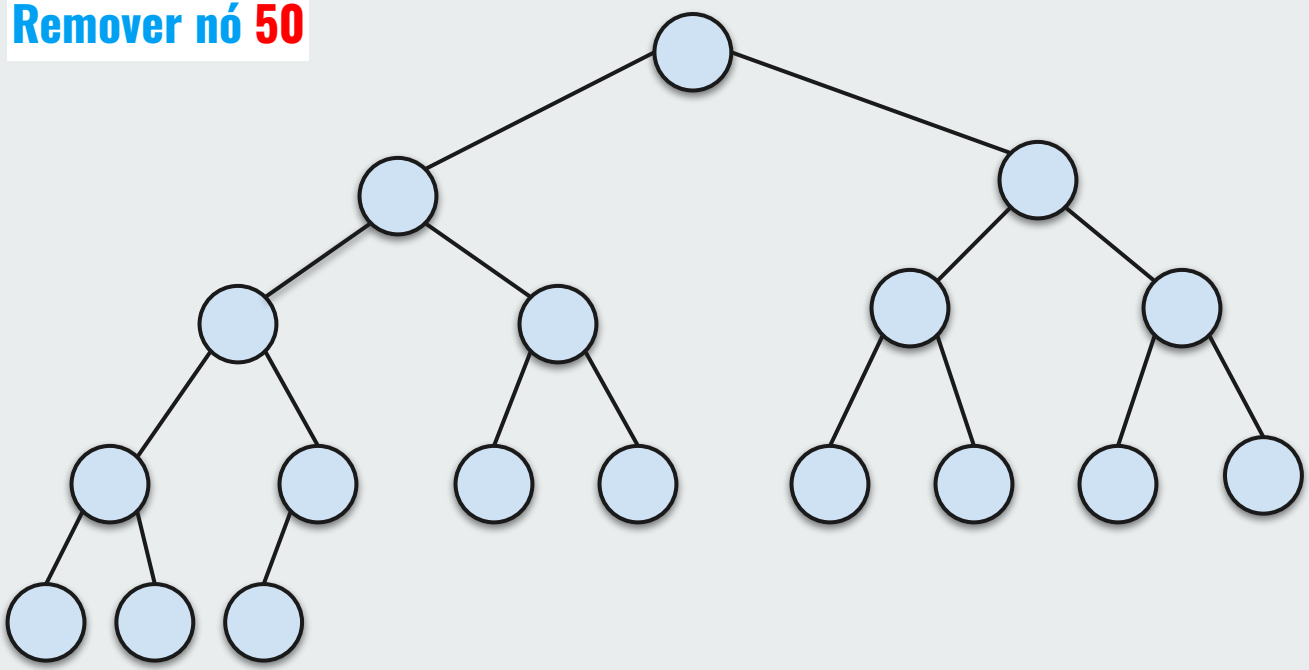
Remover nó 50



Alteração de Prioridade

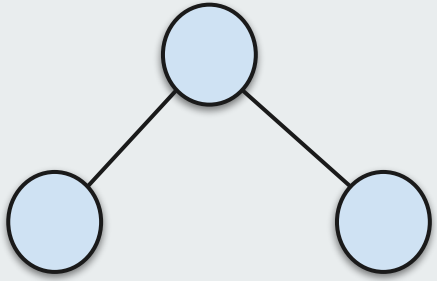
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
50	48	45	29	15	35	40	27	26	14	12	33	30	37	20	21	19	25

Remover nó 50



Pseudocódigo

```
remover(lista H){
    se (H.tamanho > 1){
       H[1] = H[H.tamanho];
       H.tamanho = H.tamanho-1;
       descer(H, 1);
    }
}
```



Operações Básicas

1. ALTERAÇÃO DE PRIORIDADE
2. INSERÇÃO
3. REMOÇÃO
4. CONSTRUÇÃO



E como está a disposição?

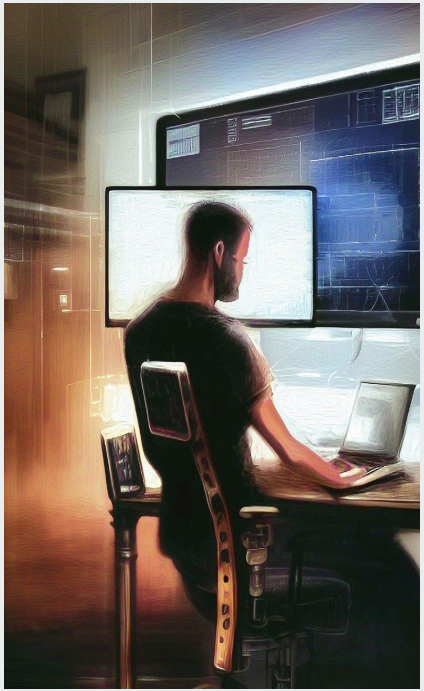


Vamos programar?



1. Implemente os procedimentos de alteração de prioridade, inserção e remoção de raiz em uma heap.





Dúvidas?!

